

DrainGuard Project Documentation

1. Project Title

****DrainGuard: Smart Urban Drainage Issue Management System****

2. Project Overview

DrainGuard is a PHP/MySQL role-based urban drainage issue management system. It digitizes drainage complaint submission, review, ward routing, maintenance assignment, proof upload, inspector verification, citizen feedback, objection handling, notifications, audit/status logs, and high-risk area detection.

The inspected project uses PHP, MySQLi, MySQL/MariaDB, HTML, CSS, JavaScript, Bootstrap, Bootstrap Icons, and XAMPP. No React, Laravel, Tailwind, PDO, or framework-based stack was found.

The system is designed for municipal drainage operations. Citizens report problems, central officers review and route complaints, ward officers verify local responsibility and assign teams, maintenance teams perform the work, and inspectors verify completion before closure.

3. Problem Statement

Manual drainage complaint management creates these problems:

- Slow complaint handling because reports are forwarded manually.
- No clear real-time tracking for citizens.
- Weak accountability because responsible users and decisions are not recorded in a structured way.
- Difficulty routing complaints to the correct city corporation, thana, ward, and area.
- Difficulty verifying actual work completion without uploaded proof and inspection.
- Lack of structured notification across citizen, central, ward, maintenance, and inspector roles.
- Lack of audit/status logs for judging progress and responsibility.

DrainGuard solves these problems by storing the full complaint lifecycle in a relational database and by giving every role a clear workflow.

4. Project Objective

The main objectives are:

- Digital complaint submission.
- Real-time complaint tracking through database statuses and logs.
- Role-wise workflow and authorization.
- Correct routing by city corporation, ward, area, location, and drain.
- Maintenance team assignment and change support.
- Proof-based work completion using image/video uploads.
- Inspector verification before final closure.
- Citizen feedback and objection after closure.
- Role-wise notification delivery.
- High-risk area detection from complaint frequency, affected area, urgency, and repeated

reports.

- Audit/status logging for accountability.

5. User Roles

| Role | What the role does |

| --- | --- |

| Citizen | Registers, submits complaints, views public board, tracks own complaints, gives feedback, and raises objections. |

| Central Officer | Accepts/rejects submitted complaints, routes accepted complaints to wards, manages officers/teams/wards/drains, and views reports and high-risk zones. |

| Ward Officer | Verifies ward complaints, rejects or marks duplicate complaints, assigns or changes maintenance teams, handles support requests, reviews objections, and manages disputed/reopened cases. |

| Maintenance Team / Maintenance Member | Receives assigned work, starts work, requests support, uploads completion proof, and views task history/feedback. |

| Inspector | Reviews solved-by-team cases, checks proof, approves work, marks false completion, and records inspection decisions. |

6. Full Workflow Explanation

Citizen

- Citizen submits a complaint from 'pages/citizen/submit-complaint.php'.
- 'auth/submit_complaint_process.php' inserts the complaint into 'complaints' with status 'submitted'.
- Uploaded complaint images/videos are stored in 'assets/uploads/complaints/' and recorded in 'complaint_media'.
- A status record is inserted in 'complaint_status_logs'.
- Central officers receive rows in 'central_notifications'.
- Citizen-facing visibility is provided through 'pages/citizen/public-board.php', 'pages/citizen/my-complaints.php', and 'pages/citizen/track-complaint.php'.

Central Officer

- Submitted complaints appear in 'pages/central/complaints.php'.
- The Central Officer accepts or rejects a complaint.
- Accepted complaints move to 'received'.
- Rejected complaints are recorded in 'complaint_decisions' and move to 'rejected_by_central'.
- Accepted complaints go to 'pages/central/routing-assignment.php'.
- Routing inserts 'complaint_assignments' with 'assignment_status = 'ward_assigned'' and changes complaint status to 'pending_verification'.
- Citizens and ward officers receive notification rows.
- Accepted/rejected records remain visible in 'pages/central/processed-complaints.php'.

Ward Officer

- Routed complaints appear in 'pages/ward/verification-queue.php'.
- Ward Officer can verify, reject, or mark duplicate.
- Verified complaints move to 'verified_by_ward'.
- Verified complaints move to 'pages/ward/local-team-assignment.php' for local team assignment.
- Assignment updates 'complaint_assignments.maintenance_team_id', 'assignment_status',

'deadline_at', 'assignment_priority', and 'task_note'.

- The complaint moves to 'team_assigned'.
- Team change logic exists in 'includes/ward/team_workflow_helpers.php'.
- Support requests are handled through 'maintenance_support_requests' and 'pages/ward/reply_support.php'.
- Citizen objections and disputed cases are handled in 'pages/ward/citizen-objections.php', 'pages/ward/reopened-disputed.php', 'pages/ward/reopened-cases.php', and 'pages/ward/false-completion-reviews.php'.

Maintenance Team

- Assigned work appears in 'pages/maintenance/assigned-tasks.php'.
- The team starts work; complaint and assignment can move to 'in_progress'.
- Active work appears in 'pages/maintenance/in-progress-work.php'.
- Support can be requested through 'notifications/send_maintenance_support_request.php', which inserts 'maintenance_support_requests' and notifies the Ward Officer.
- Completion proof is uploaded from 'pages/maintenance/upload-completion-proof.php'.
- Proof files are stored in 'assets/uploads/maintenance_proofs/' and metadata is inserted into 'maintenance_proofs'.
- The assignment is marked 'completed', the complaint moves toward 'solved_by_team' / inspector review, and inspectors receive notification.

Inspector

- Solved cases appear in 'pages/inspector/solved-cases.php' and inspection pages.
- Inspector reviews before/after details in 'pages/inspector/before-after-review.php' and decisions in 'pages/inspector/inspection-queue.php'.
- Approval inserts 'inspection_logs', changes the complaint to 'closed', accepts proof, and notifies related roles.
- False completion inserts 'inspection_logs', changes the complaint to 'disputed', and sends it back for ward dispute handling.

Feedback / Objection / Dispute

- Citizen feedback and objections are submitted from 'pages/citizen/feedback-reopen.php'.
- Normal feedback inserts into 'feedbacks' with 'feedback_type = 'feedback'' and may create 'maintenance_team_reviews'.
- Bad feedback/objection inserts 'feedbacks' with 'feedback_type = 'false_completion'', inserts 'reopen_requests', and changes the complaint to 'disputed'.
- Ward, central, inspector, and maintenance notification rows are created where recipient data exists.
- Valid objection/dispute can reopen or reassign a complaint; invalid objection can be rejected/resolved.

A main strength of the project is real-time complaint tracking through status changes and the role-wise notification system.

7. Core Features

| Feature | Actual evidence in project |

| --- | --- |

| Login and role-based redirection | 'auth/login.php', 'auth/login_process.php',

'auth/session_check.php' |
 | Citizen registration | 'citizenRegistration/citizen_signup.php' |
 | Complaint submission | 'pages/citizen/submit-complaint.php',
 'auth/submit_complaint_process.php' |
Public complaint board	'pages/citizen/public-board.php'
My complaints	'pages/citizen/my-complaints.php'
Complaint tracking	'pages/citizen/track-complaint.php', 'complaint_status_logs'
Central review	'pages/central/complaints.php'
Ward routing	'pages/central/routing-assignment.php'
Ward verification	'pages/ward/verification-queue.php'
Team assignment/change	'pages/ward/local-team-assignment.php',
'includes/ward/team_workflow_helpers.php'	
Assigned work and start work	'pages/maintenance/assigned-tasks.php',
'pages/maintenance/in-progress-work.php'	
Support request	'notifications/send_maintenance_support_request.php',
'maintenance_support_requests'	
Completion proof	'pages/maintenance/upload-completion-proof.php', 'maintenance_proofs'
Inspector verification	'pages/inspector/inspection-queue.php',
'pages/inspector/before-after-review.php'	
False completion handling	'inspection_logs', 'false_completion_reviews', ward dispute pages
Feedback / objection / reopen	'pages/citizen/feedback-reopen.php', 'feedbacks',
'reopen_requests'	
Role notifications	Five notification tables and role notification pages
Audit/status logs	'complaint_status_logs', 'complaint_decisions', 'inspection_logs'
Risk zones	'risk', central/ward/citizen high-risk pages
Role instructions	'role_instructions', ward/inspector instruction details pages
Comment/reply/like system	'commentSystem/', 'comment_likes'

8. Database Design

The database schema is defined in 'database/drainguard.sql'. It uses InnoDB tables, primary keys, foreign keys, indexed lookup columns, enum workflow statuses, and timestamp columns.

User / Login Section

Table	Purpose	Important columns	Relationship
users	Central authentication table.	'user_id', 'user_name', 'user_mail', 'user_password', 'user_role', 'user_status', 'login_access', 'last_active', 'reset_token', 'reset_time'	Parent login entity for all roles and notification recipients.
citizens	Citizen profile table.	'citizen_id', 'user_id', 'full_name', 'phone_number', 'user_mail', address fields, 'profile_photo', demerit/suspension fields	'user_id' references 'users'.
central_officers	Central officer profile.	'central_officer_id', 'user_id', 'full_name', 'phone', 'employee_code', 'designation', 'office_address'	Extends 'users'.
ward_officers	Ward officer profile and assigned ward.	'ward_officer_id', 'user_id', 'city_cor_id', 'assigned_ward_id', 'full_name', 'designation'	Connects a user to one city corporation and ward.
maintenance_teams	Maintenance team master data.	'maintenance_team_id', 'team_name', 'city_cor_id', 'anchal_id', 'availability_status', 'assistant_login_access'	Used by assignments, members, proofs, reviews.

| 'maintenance_team_members' | Maintenance staff. | 'member_id', 'maintenance_team_id', 'user_id', 'full_name', 'role', 'status', demerit/warning fields | Connects members to teams and optional login users. |

| 'inspectors' | Inspector profile and assigned ward. | 'inspector_id', 'user_id', 'city_cor_id', 'assigned_ward_id', 'full_name', suspension fields | Used in inspection decisions. |

Location Section

| Table | Purpose | Important columns | Relationship |

| --- | --- | --- | --- |

| 'cities' | City lookup. | 'city_id', 'city_name' | Parent for city corporations. |

| 'city_corporations' | City corporation lookup. | 'city_cor_id', 'city_id', 'city_cor_name' | Used by locations/officers/teams. |

| 'thanas' | Thana lookup. | 'thana_id', 'city_cor_id', 'thana_name' | Parent for wards and location selection. |

| 'anchals' | Administrative zone lookup. | 'anchal_id', 'city_cor_id', 'anchal_name' | Used by wards and maintenance teams. |

| 'wards' | Ward lookup. | 'ward_id', 'city_cor_id', 'thana_id', 'anchal_id', 'ward_no', 'ward_name' | Used for routing and ward officer assignment. |

| 'areas' | Area lookup under ward. | 'area_id', 'ward_id', 'area_name' | Used by 'locations'. |

| 'locations' | Normalized location combination. | 'loc_id', 'city_id', 'city_cor_id', 'thana_id', 'ward_id', 'area_id' | Connected to complaints and drains. |

| 'drains' | Drain record and condition. | 'drain_id', 'loc_id', 'drain_code', 'drain_name', 'drain_address_description', 'drain_condition', condition update metadata | Connected to complaints through 'drain_id'. |

Complaint Core Section

| Table | Purpose | Important columns | Relationship |

| --- | --- | --- | --- |

| 'complaints' | Main transaction table. | 'complaint_id', 'complaint_code', 'user_id', 'loc_id', 'drain_id', 'issue_id', 'affected_area_id', descriptions, 'complaint_status', 'work_started_at', 'parent_complaint_id', 'is_repeat_complaint', timestamps | References user, location, drain, issue, affected area, and optional parent complaint. |

| 'complaint_media' | Complaint images/videos. | 'media_id', 'complaint_id', 'media_type', 'media_path', 'original_name', file metadata | References 'complaints'. |

| 'complaint_status_logs' | Status/audit history. | 'log_id', 'complaint_id', 'old_status', 'new_status', 'action_by_user_id', 'action_by_role', 'remarks', 'created_at' | References complaints/users. |

| 'complaint_decisions' | Rejection/duplicate/final decision reasons. | 'decision_id', 'complaint_id', 'decided_by_user_id', 'decided_by_role', 'decision_type', 'reason', 'reference_complaint_id' | References complaints/users. |

| 'issues' | Issue lookup. | 'issue_id', 'issue_name', 'priority' | Used by complaints and risk logic. |

| 'affected_areas' | Affected area lookup. | 'affected_area_id', 'affected_area_name', 'priority' | Used by complaints and urgency logic. |

| 'comment_likes' | Discussion comments, replies, likes, dislikes. | 'id', 'complaint_id', 'parent_id', 'user_id', 'type', 'comment_text', 'is_deleted' | Supports 'commentSystem/'. |

Assignment and Maintenance Section

| Table | Purpose | Important columns | Relationship |

| --- | --- | --- | --- |

| 'complaint_assignments' | Ward/team assignment table. | 'assignment_id', 'complaint_id', 'ward_id', 'maintenance_team_id', 'assigned_by', 'assignment_status', 'assigned_at', 'deadline_at', 'assignment_priority', 'task_note' | Connects complaint to ward, central assigner, and maintenance team. |

| 'maintenance_proofs' | Completion proof table. | 'proof_id', 'assignment_id', 'complaint_id', 'maintenance_team_id', 'uploaded_by', 'proof_stage', 'media_type', 'media_path', 'proof_note', 'proof_status' | References assignment, complaint, team, and uploader. |

| 'maintenance_support_requests' | Support requests from team to ward officer. | 'support_request_id', 'assignment_id', 'complaint_id', 'maintenance_team_id', 'requested_by', 'ward_officer_user_id', 'support_reason', 'request_status', 'ward_reply' | Connects maintenance issue with ward response. |

| 'maintenance_updates' | Maintenance work update history. | 'update_id', 'complaint_id', 'assignment_id', 'maintenance_team_id', 'updated_by_user_id', 'work_status' | Tracks work progress. |

| 'delay_requests' | Delayed task support. | 'delay_request_id', assignment/complaint/team/user fields | Used by delayed task pages. |

Inspection / Reopen / Feedback Section

| Table | Purpose | Important columns | Relationship |

| --- | --- | --- | --- |

| 'inspection_logs' | Inspector decision history. | 'log_id', 'complaint_id', 'assignment_id', 'inspector_user_id', 'decision_type', 'decision_note', 'source_type', 'source_id' | References complaint, assignment, inspector user. |

| 'false_completion_reviews' | Ward review of false completion cases. | 'review_id', complaint/assignment/team/review fields | Supports false completion handling. |

| 'reopen_requests' | Citizen objection/reopen/dispute requests. | 'reopen_id', 'complaint_id', 'requested_by', 'request_type', 'reason', 'request_status', 'handled_by', timestamps | References complaint and users. |

| 'feedbacks' | Citizen feedback and objection feedback. | 'feedback_id', 'complaint_id', 'user_id', 'rating', 'feedback_text', 'feedback_type', 'created_at' | References complaint/user. |

| 'maintenance_team_reviews' | Citizen review of maintenance team. | 'review_id', 'complaint_id', 'citizen_user_id', 'maintenance_team_id', rating/review fields | Used by team performance pages. |

| 'objection_reviews' | Review records for objections. | 'review_id', 'reopen_id', 'complaint_id', 'reviewed_by', 'reviewer_role', 'review_action' | Supports objection review tracking. |

Risk and Instruction Section

| Table | Purpose | Important columns | Relationship |

| --- | --- | --- | --- |

| 'risk' | High-risk area records. | 'risk_id', 'risk_area_key', location ids, 'urgency_level', 'risk_status', complaint count fields, 'last_complaint_id' | References last related complaint and location ids. |

| 'role_instructions' | Officer-to-role instructions. | 'instruction_id', 'sender_user_id', 'receiver_user_id', 'receiver_role', 'ward_id', 'instruction_title', 'instruction_message', 'priority', 'instruction_status' | Used by instruction detail pages. |

| 'instruction_notifications_map' | Instruction notification mapping. | 'map_id' and mapping

fields | Supports instruction notifications. |

Notification Section

All role notification tables share this structure: 'notification_id', 'recipient_user_id', 'sender_user_id', 'related_complaint_id', 'notification_type', 'notification_title', 'notification_message', 'is_read', and 'created_at'.

Table	Recipient role
---	---
'citizen_notifications'	Citizen
'central_notifications'	Central Officer
'ward_notifications'	Ward Officer
'maintenance_notifications'	Maintenance Team / Maintenance Member
'inspector_notifications'	Inspector

9. ER Diagram Explanation

- 'users' is the central login entity.
- Role tables extend 'users' with role-specific attributes.
- 'complaints' is the core transaction entity.
- 'locations', 'drains', 'issues', and 'affected_areas' classify each complaint.
- 'complaint_assignments' connects complaints to wards and maintenance teams.
- 'maintenance_proofs' and 'inspection_logs' verify work completion.
- 'feedbacks', 'reopen_requests', 'false_completion_reviews', and 'objection_reviews' manage post-closure review.
- Notification tables connect workflow events to recipient users.
- 'complaint_status_logs' and 'complaint_decisions' maintain audit evidence.
- 'risk' supports high-risk area detection.

10. Relational Schema Explanation

- 'users.user_id' connects to 'citizens', 'central_officers', 'ward_officers', 'inspectors', and 'maintenance_team_members'.
- 'complaints.user_id' connects the complaint to the citizen/login user.
- 'complaints.loc_id' connects to 'locations.loc_id'.
- 'complaints.drain_id' connects to 'drains.drain_id'.
- 'complaints.issue_id' connects to 'issues.issue_id'.
- 'complaints.affected_area_id' connects to 'affected_areas.affected_area_id'.
- 'complaints.parent_complaint_id' connects repeat/duplicate complaints to an earlier complaint.
- 'complaint_assignments.complaint_id' connects assignment with complaint.
- 'complaint_assignments.maintenance_team_id' connects assignment with team.
- 'maintenance_proofs.assignment_id' and 'maintenance_proofs.complaint_id' connect proof with assignment and complaint.
- 'inspection_logs.complaint_id', 'inspection_logs.assignment_id', and 'inspection_logs.inspector_user_id' connect inspector decision with complaint, assignment, and inspector user.
- Notification tables connect 'recipient_user_id', 'sender_user_id', and 'related_complaint_id' to users and complaints.

11. Normalization

- **1NF:** Columns are atomic. Complaint status, issue id, affected area id, location id, media path, proof path, and notification fields are stored separately.
- **2NF:** Non-key attributes depend on the table primary key. For example, complaint descriptions depend on 'complaint_id', and proof file data depends on 'proof_id'.
- **3NF:** Users, roles, complaints, media, logs, assignments, proofs, locations, notifications, feedback, and risk data are separated into dedicated tables.
- The schema is mainly normalized up to **3NF**.
- **BCNF** can be discussed for lookup tables such as 'issues', 'affected_areas', 'cities', 'wards', and 'areas'.

12. Advanced DBMS Topics Used

1. Transaction Management

Where used it: Complaint submission, central routing, ward verification, maintenance proof upload, support request, and inspector decision workflows.

Table used: 'complaints', 'complaint_media', 'complaint_status_logs', 'central_notifications', 'complaint_assignments', 'maintenance_proofs', 'inspection_logs'.

Query / code snippet:

```
““php
mysqli_begin_transaction($conn);
// dependent INSERT and UPDATE statements
mysqli_commit($conn);
// on error
mysqli_rollback($conn);
““
```

2. ACID Properties

Where used it: Multi-step complaint submission and proof/inspection updates are handled as one unit so partial database changes are avoided.

Table used: 'complaints', 'complaint_status_logs', 'complaint_media', 'maintenance_proofs', 'inspection_logs', 'reopen_requests'.

Query / code snippet:

```
““php
try {
mysqli_begin_transaction($conn);
// insert complaint, log, notification, proof, inspection, or reopen data
mysqli_commit($conn);
} catch (Throwable $exception) {
mysqli_rollback($conn);
}
““
```

3. Commit and Rollback

****Where used it:**** 'auth/submit_complaint_process.php', 'pages/central/routing-assignment.php', 'pages/ward/verification-queue.php', 'pages/maintenance/upload-completion-proof.php', and 'pages/inspector/inspection-queue.php'.

****Table used:**** 'complaints', 'complaint_assignments', 'maintenance_proofs', 'inspection_logs', notification tables.

****Query / code snippet:****

```
““php
mysqli_commit($conn);
mysqli_rollback($conn);
““
```

4. Prepared Statement

****Where used it:**** Login, complaint submission, assignment, proof upload, feedback, notifications, search/filter pages, and reports.

****Table used:**** 'users', 'complaints', 'complaint_assignments', 'feedbacks', notifications, and many other tables.

****Query / code snippet:****

```
““php
$stmt = mysqli_prepare($conn, "SELECT user_status, login_access FROM users WHERE user_id = ?");
mysqli_stmt_bind_param($stmt, "i", $userId);
mysqli_stmt_execute($stmt);
““
```

5. Query Processing using JOIN, filtering, and subquery

****Where used it:**** Complaint routing, tracking, inspector review, reports, high-risk zones, and role dashboards.

****Table used:**** 'complaints', 'locations', 'wards', 'city_corporations', 'complaint_assignments', 'maintenance_teams', 'issues', 'areas'.

****Query / code snippet:****

```
““sql
SELECT c.complaint_id, c.complaint_status, ca.assignment_id, ca.maintenance_team_id
FROM complaints c
INNER JOIN complaint_assignments ca ON ca.complaint_id = c.complaint_id
WHERE c.complaint_id = ?
AND ca.ward_id = ?
AND c.complaint_status = 'inspector_verification'
LIMIT 1;
““
```

6. Indexing for faster lookup

****Where used it:**** Frequent lookup by complaint, user, location, team, ward, notification recipient, type, read flag, and created time.

****Table used:**** 'complaints', 'complaint_assignments', 'complaint_status_logs', 'maintenance_proofs', notification tables, 'inspection_logs', 'locations', 'risk'.

****Query / code snippet:****

```
““sql
ALTER TABLE 'complaints'
ADD KEY 'idx_complaint_user' ('user_id'),
ADD KEY 'idx_complaint_location' ('loc_id'),
ADD KEY 'fk_complaints_drain' ('drain_id');
““
```

7. Database Security

****Where used it:**** Password hashing, prepared statements, login access checks, user status checks, reset tokens, and session protection.

****Table used:**** 'users', 'remember_tokens', role profile tables.

****Query / code snippet:****

```
““php
$hashedPassword = password_hash($newPassword, PASSWORD_DEFAULT);
$updateSql = "UPDATE users SET user_password = ?, reset_token = NULL, reset_time = NULL WHERE
user_mail = ?";
““
```

8. Authentication

****Where used it:**** Login validates identity through email/phone, password hash, account status, login access, and remember token.

****Table used:**** 'users', 'remember_tokens'.

****Query / code snippet:****

```
““php
if (!password_verify($password, $user["user_password"])) {
// invalid login
}
““
```

9. Authorization / Role-based access

****Where used it:**** Role pages check session role before allowing access.

****Table used:**** 'users', role tables.

****Query / code snippet:****

```

““php
if (!isset($_SESSION["user_role"]) || $_SESSION["user_role"] !== "ward_officer") {
header("Location: ../../index.php");
exit();
}
””

```

10. Audit log / Status log

****Where used it:**** Complaint submission, central decisions, ward verification, assignment, maintenance progress, inspection, reopen, and dispute workflows.

****Table used:**** 'complaint_status_logs', 'inspection_logs', 'complaint_decisions'.

****Query / code snippet:****

```

““sql
INSERT INTO complaint_status_logs
(complaint_id, old_status, new_status, action_by_user_id, action_by_role, remarks, created_at)
VALUES (?, ?, ?, ?, 'central_officer', ?, NOW());
””

```

13. SQL Query Demonstration

INSERT complaint

Real pattern from 'auth/submit_complaint_process.php':

```

““sql
INSERT INTO complaints (
complaint_code, user_id, loc_id, drain_id, issue_id, affected_area_id,
address_description, problem_description, complaint_status,
parent_complaint_id, is_repeat_complaint
)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'submitted', ?, ?);
””

```

SELECT complaint with JOIN

Real pattern from 'pages/ward/verification-queue.php':

```

““sql
SELECT c.complaint_id, c.complaint_code, c.user_id AS citizen_user_id,
c.complaint_status, l.ward_id, ca.assigned_by AS central_user_id
FROM complaints c
INNER JOIN locations l ON c.loc_id = l.loc_id
LEFT JOIN complaint_assignments ca ON c.complaint_id = ca.complaint_id
WHERE c.complaint_id = ?
AND c.complaint_status = 'pending_verification'
AND l.ward_id = ? AND l.city_cor_id = ?
ORDER BY ca.assignment_id DESC

```

LIMIT 1;

““

UPDATE status

Real pattern from central routing and other workflow pages:

““sql

UPDATE complaints

SET complaint_status = 'pending_verification', updated_at = NOW()

WHERE complaint_id = ?;

““

INSERT status log

Real pattern from central complaint helper logic:

““sql

INSERT INTO complaint_status_logs

(complaint_id, old_status, new_status, action_by_user_id, action_by_role, remarks, created_at)

VALUES (?, ?, ?, ?, 'central_officer', ?, NOW());

““

INSERT notification

Real pattern from routing and notification pages:

““sql

INSERT INTO citizen_notifications

(recipient_user_id, sender_user_id, related_complaint_id, notification_type,

notification_title, notification_message, is_read, created_at)

VALUES (?, ?, ?, 'status_update', 'Complaint Routed to Ward', ?, 0, NOW());

““

Assignment query

Real pattern from 'pages/central/routing-assignment.php':

““sql

INSERT INTO complaint_assignments

(complaint_id, ward_id, assigned_by, assignment_status)

VALUES (?, ?, ?, 'ward_assigned');

““

Maintenance proof query

Real pattern from 'pages/maintenance/upload-completion-proof.php':

““sql

INSERT INTO maintenance_proofs

(assignment_id, complaint_id, maintenance_team_id, uploaded_by, proof_stage,

media_type, media_path, original_name, file_size, mime_type, proof_note, proof_status)

```
VALUES (?, ?, ?, ?, 'after', ?, ?, ?, ?, ?, 'submitted');
```

```
““
```

Inspector verification query

Real pattern from 'pages/inspector/inspection-queue.php':

```
““sql
```

```
INSERT INTO inspection_logs  
(complaint_id, assignment_id, inspector_user_id, decision_type, decision_note, source_type)  
VALUES (?, ?, ?, ?, ?, 'inspection_queue');
```

```
““
```

Feedback query

Real pattern from 'pages/citizen/feedback-reopen.php':

```
““sql
```

```
INSERT INTO feedbacks  
(complaint_id, user_id, rating, feedback_text, feedback_type)  
VALUES (?, ?, ?, ?, 'feedback');
```

```
““
```

Search/filter query

Search/filter inputs exist in pages such as inspector review, inspection queue, citizen board, tracking, and report helpers. A representative filtered query is:

```
““sql
```

```
SELECT c.complaint_id, c.complaint_status, ca.assignment_id, ca.maintenance_team_id  
FROM complaints c  
INNER JOIN complaint_assignments ca ON ca.complaint_id = c.complaint_id  
WHERE c.complaint_id = ?  
AND ca.ward_id = ?  
AND c.complaint_status = 'inspector_verification'  
LIMIT 1;
```

```
““
```

14. Security Implementation

Security mechanisms found in the project include:

- Password hashing with 'password_hash(..., PASSWORD_DEFAULT)'.
- Password validation with 'password_verify()'.
- Central session checking in 'auth/session_check.php'.
- 'login_access' in 'users', allowing login to be enabled/disabled without deleting users.
- 'user_status' in 'users', supporting 'active', 'inactive', 'suspended', and 'banned'.
- Role-based page access using session role checks and '\$allowed_role' / '\$allowed_roles'.
- Prepared statements for dynamic SQL queries.
- Forgot password/reset token support through 'auth/forgot_password.php', 'auth/reset_password.php', 'users.reset_token', and 'users.reset_time'.
- Remember-me token support through 'remember_tokens' with hashed validators.

- No plain-text password storage was found; database user passwords are bcrypt-style hashes.

15. Notification System

DrainGuard uses separate role-wise notification tables:

- 'citizen_notifications'
- 'central_notifications'
- 'ward_notifications'
- 'maintenance_notifications'
- 'inspector_notifications'

Each notification stores recipient user, sender user, related complaint, notification type, title, message, read/unread status, and creation time. Notification pages exist for every major role:

- 'pages/citizen/notifications.php'
- 'pages/central/notifications.php'
- 'pages/ward/notifications.php'
- 'pages/maintenance/notifications.php'
- 'pages/inspector/notifications.php'

'includes/notification_workflow_cleanup.php' is used to reduce duplicate workflow notifications. Notification records should redirect users to the related complaint/workflow page where applicable. Supporting files include 'css/global/notification-target.css' and 'js/global/notification-target.js'.

16. Real-time Tracking

The system tracks complaint progress using:

- 'complaints.complaint_status'
- 'complaint_status_logs'
- Role notification tables
- Citizen tracking page 'pages/citizen/track-complaint.php'

Important statuses include 'submitted', 'received', 'pending_verification', 'verified_by_ward', 'team_assigned', 'in_progress', 'solved_by_team', 'inspector_verification', 'closed', 'reopened', 'disputed', 'duplicate', and rejection statuses. The inspected files indicate database-driven tracking with refresh/navigation or polling-style reads, not WebSocket live push.

17. Risk / High-risk Area Detection

Risk logic is implemented through 'risk' and complaint submission logic in 'auth/submit_complaint_process.php'. The system considers:

- Complaint count in the last 7 days.
- Complaint count in the last 30 days.
- Complaint count in the current week.
- Issue priority from 'issues'.
- Affected area priority from 'affected_areas'.

- Repeated complaint relationships using 'parent_complaint_id' and 'is_repeat_complaint'.

The code explicitly treats **Water Contamination** as high/emergency priority. Risk pages exist in:

- 'pages/central/high-risk-zones.php'
- 'pages/ward/ward-risk-zones.php'
- 'pages/citizen/high-risk-areas.php'

18. File/Folder Structure

Folder/File	Responsibility
---	---
'auth/'	Login, logout, session checks, forgot/reset password, complaint submission process, report generation.
'pages/citizen/'	Citizen dashboard, complaint submission, public board, my complaints, tracking, feedback/reopen, objection, high-risk areas, discussion, notifications, profile/settings.
'pages/central/'	Central dashboard, complaint review, processed complaints, routing, user management, ward/area/drain management, high-risk zones, reports, team feedback, discussion, notifications.
'pages/ward/'	Ward dashboard, verification queue, local team assignment, in-progress cases, citizen objections, reopened/disputed cases, false completion reviews, support replies, reports, notifications.
'pages/maintenance/'	Maintenance dashboard, assigned tasks, in-progress work, completion proof upload, delayed tasks, drain reference, feedback, task history, notifications, profile/settings.
'pages/inspector/'	Inspector dashboard, solved cases, before/after review, inspection queue, inspection logs, false completion reports, citizen objections, notifications, profile.
'includes/'	Shared helpers, sidebars, topbars, notification cleanup, reports, ward team workflow helpers, maintenance access control.
'css/'	Global and role-wise styles.
'js/'	Global and role-wise JavaScript.
'notifications/'	Notification-related process scripts such as maintenance support request.
'commentSystem/'	Add/fetch/delete comments, replies, reactions, and discussion logic.
'assets/uploads/'	Uploaded profile photos, complaint media, and maintenance proof media.
'assets/reports/'	Generated central and ward reports.
'database/'	SQL dump/schema file 'drainguard.sql'.

19. Important Pages and Responsibilities

Citizen Pages

Page	Responsibility
---	---
'dashboard.php'	Citizen overview.
'submit-complaint.php'	Complaint submission form.
'public-board.php'	Public complaint board.
'my-complaints.php'	Citizen's own complaints.
'track-complaint.php'	Complaint tracking and progress view.
'feedback-reopen.php'	Feedback and citizen objection.
'citizen-objection.php'	Objection-related citizen page.

'high-risk-areas.php'	Citizen risk-zone view.
'discussion.php'	Complaint discussion/comments.
'notifications.php'	Citizen notifications.

Central Pages

Page	Responsibility
'complaints.php'	Review submitted complaints.
'processed-complaints.php'	View processed complaints.
'routing-assignment.php'	Route accepted complaints to wards.
'user-management.php'	Manage users.
'add-ward-officer.php', 'add-inspector.php', 'add-maintenance-team.php', 'add-team-member.php'	Create officer/team/member records.
'ward-area.php'	Ward and area management.
'drain-records.php'	Drain records and condition management.
'high-risk-zones.php'	System-wide risk zones.
'reports.php'	Central reports.
'team-feedback.php'	Maintenance team feedback/performance.

Ward Pages

Page	Responsibility
'verification-queue.php'	Verify, reject, or mark duplicate complaints.
'local-team-assignment.php'	Assign/change maintenance teams.
'in-progress-cases.php'	Monitor active ward work.
'reply_support.php'	Reply to maintenance support requests.
'citizen-objections.php'	Review citizen objections.
'reopened-cases.php'	Manage reopened cases.
'reopened-disputed.php'	Handle disputed/reopened workflow.
'false-completion-reviews.php'	Review inspector false-completion reports.
'ward-risk-zones.php'	Ward-level high-risk zones.
'local-reports.php'	Ward reports.

Maintenance Pages

Page	Responsibility
'assigned-tasks.php'	View assigned complaints and start work.
'in-progress-work.php'	Manage active work.
'upload-completion-proof.php'	Submit completion proof.
'delayed-tasks.php'	View delayed tasks.
'task-history.php'	View task history.
'drain-area-reference.php'	Drain/location reference.
'feedback.php'	View maintenance feedback.

Inspector Pages

Page	Responsibility
'solved-cases.php'	View cases solved by maintenance team.

'before-after-review.php'	Compare complaint and completion proof.
'inspection-queue.php'	Approve work or mark false completion.
'inspection-logs.php'	View inspection history.
'false-completion-reports.php'	View false completion reports.
'citizen-objections.php'	Review citizen objection cases.

20. Testing / Demo Data

The SQL dump contains development/demo users, complaints, notifications, reports, and upload paths. The system also has role creation pages for adding test officers, teams, and members.

A judge can test the project with this flow:

1. Login as a citizen.
2. Submit a complaint with location, issue, affected area, description, and optional media.
3. Check Public Complaint Board, My Complaints, Track Complaint, and citizen notifications.
4. Login as Central Officer.
5. Accept the complaint and route it to a ward.
6. Login as the assigned Ward Officer.
7. Verify the complaint and assign a maintenance team.
8. Login as maintenance team leader/member.
9. Start work, optionally request support, and upload completion proof.
10. Login as Inspector.
11. Review before/after proof and approve work or mark false completion.
12. Login again as Citizen.
13. Submit feedback or objection.
14. Check notifications and complaint tracking after each step.

21. Limitations

- The project is local unless deployed and configured on a live server.
- No GPS/map integration was found in the inspected files.
- Notifications are system/database-based, not SMS-based.
- Email support is present for password reset through PHPMailer, but workflow notifications are mainly in-system.
- Real-time tracking may require refresh or polling unless a WebSocket layer is added later.
- Risk detection is rule/frequency-based, not AI-based.
- Analytics can be expanded further.

22. Future Scope

- GIS/map integration.
- SMS/email notification for workflow events.
- Mobile app for citizens and field workers.
- AI-based complaint classification.
- Automated risk prediction.
- Analytics dashboard.
- Deployment on live server.
- Role-wise performance reports.
- SLA/deadline escalation and automatic reminders.

23. Conclusion

DrainGuard is useful because it is transparent, accountable, role-based, and database-driven. It tracks the full complaint lifecycle from citizen submission to central review, ward routing, maintenance assignment, proof upload, inspector verification, citizen feedback, objection handling, and risk analysis.

For a DBMS/project-show judge, DrainGuard demonstrates normalized relational design, primary/foreign key relationships, indexes, prepared statements, transactions, commit/rollback, authentication, authorization, audit logs, notification tables, and a complete multi-role workflow.